Week 4: Count, Binary, and Categorical Outcomes

Ronnie Bailey-Steinitz

2026-07-17

Skills Learning – Lecture

Last week, we introduced nonparametric tests and began working with count data.

This week, we will complete several major branches of our model-selection framework by asking:

> **What analysis should I use when my response variable is a count, binary, or categorical variable?**

Today we will focus on:

- negative binomial regression for count outcomes
- logistic regression for binary outcomes
- chi-square tests for relationships between categorical variables
- Fisher's exact test when expected counts are small

0. Load Required Packages

```r
library(tidyverse)
library(janitor)
library(here)
library(MASS)
```

1. Load and Preview the Dataset

We will continue using the faux gorilla hormone and behavioral dataset.

```r
data <- readr::read_csv(
  here::here("Week 3/gorilla_hormone_data_week3.csv")
) %>%
  janitor::clean_names()
```

```
## Rows: 240 Columns: 21
## -- Column specification ------------------------------------
## Delimiter: ","
## chr  (9): sample_id, gorilla_id, gorilla_name, group, sa...
## dbl (12): age_years, dominance_rank, group_size, fruit_a...
```

```
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
glimpse(data)
```

```
## Rows: 240
## Columns: 21
## $ sample_id                <chr> "G001_S01", "G001_S02", "~
## $ gorilla_id               <chr> "G001", "G001", "G001", "~
## $ gorilla_name             <chr> "Gasore", "Gasore", "Gaso~
## $ group                    <chr> "BWE", "BWE", "BWE", "BWE~
## $ sample_date              <chr> "5/23/25", "8/18/25", "9/~
## $ observer                 <chr> "Mark", "Sarah", "Mary", ~
## $ sex                      <chr> "F", "F", "F", "F", "F", ~
## $ age_class                <chr> "adult", "adult", "adult"~
## $ age_years                <dbl> 14.2, 14.2, 14.2, 14.2, 1~
## $ dominance_rank           <dbl> 8, 8, 8, 8, 8, 8, 8, 8, 6~
## $ group_size               <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 3~
## $ fruit_availability_index <dbl> 0.4040977, 0.3301009, 0.3~
## $ feeding_time_prop        <dbl> 0.2554745, 0.6272515, 0.5~
## $ aggression_events        <dbl> 2, 0, 1, 0, 0, 1, 1, 0, 3~
## $ parasite_load            <dbl> 46, 33, 36, 49, 39, 33, 6~
## $ respiratory_signs        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ injury_status            <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 1~
## $ cortisol_ng_ml           <dbl> 19.158612, 22.606257, 28.~
## $ testosterone_ng_ml       <dbl> 5.717721, 3.666248, 3.084~
## $ injury_label             <chr> "Not injured", "Not injur~
## $ play_events              <dbl> 3, 4, 7, 4, 4, 8, 5, 2, 0~
```

Part 1: Count Outcomes

2. Count Response Variables

Count variables represent the number of times something happened.

Examples include:

- number of parasites
- number of aggression events
- number of nests
- number of vocalizations
- number of animal sightings

Count variables:

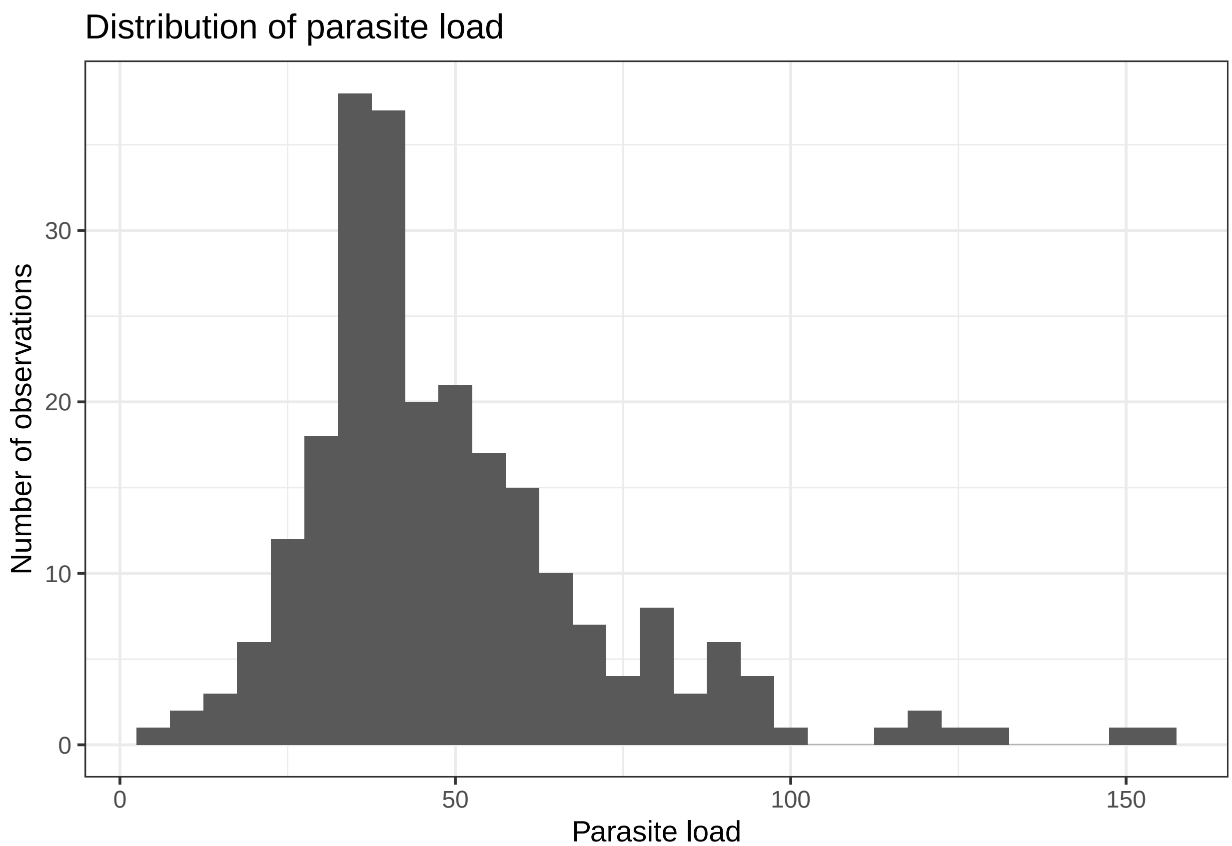
- are whole numbers
- cannot be negative
- often include many zeros
- are often highly variable in ecological datasets

Today, we will use `parasite_load` as our count response variable.

```
summary(data$parasite_load)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    6.00   35.00   43.00   49.48   59.00  153.00
```

```
ggplot(data, aes(x = parasite_load)) +
  geom_histogram(binwidth = 5) +
  labs(
    x = "Parasite load",
    y = "Number of observations",
    title = "Distribution of parasite load"
  ) +
  theme_bw()
```



3. Negative Binomial Regression

Negative binomial regression is used when:

- the response variable is a count
- the counts are highly variable
- the response cannot be negative

Biological question:

> **Does parasite load change with age?**

Response variable:

- `parasite_load`, a count

Predictor variable:

- `age_years`, continuous

```r
mod_nb <- MASS::glm.nb(
  parasite_load ~ age_years,
  data = data
)

summary(mod_nb)
```

```
##
## Call:
## MASS::glm.nb(formula = parasite_load ~ age_years, data = data,
##     init.theta = 6.325617361, link = log)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) 3.644209   0.074484  48.926  < 2e-16 ***
## age_years   0.010427   0.002848   3.661 0.000252 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(6.3256) family taken to be 1)
##
##     Null deviance: 258.05  on 239  degrees of freedom
## Residual deviance: 245.21  on 238  degrees of freedom
## AIC: 2116.5
##
## Number of Fisher Scoring iterations: 1
##
##
##               Theta:  6.326
##           Std. Err.:  0.640
##
##  2 x log-likelihood:  -2110.464
```

> We asked: Does parasite load change with age?

Negative binomial regressions use a log link, so these coefficients are on the log scale.

```
text            Estimate (Intercept) 3.644209 age_years   0.010427
```

- **Intercept:** the predicted log parasite load when `age = 0`
- **Estimate:** age = `0.0104` ... the positive sign tells us: *as age increases, parasite load tends to increase.* Notice that the coefficient is quite small.
- **p-value:** There is strong evidence that parasite load changes with age.
- **Fisher scoring iterations:** You can safely ignore this. It just tells you how many computational steps R needed to fit the model. In this case, only 1!
- **Theta:** `theta = 6.326` controls how much extra variation the model allows. This is the extra dispersion parameter that makes the negative binomial model different from the Poisson model. Theta measures how much **extra** variation the model allows beyond what a Poisson model can accommodate (with equal mean and variance). We usually don't interpret theta biologically; it simply helps the model fit highly variable count data.

... now, let's exponentiate the result:

3.1 Interpret the Coefficients

Negative binomial regression uses a log link, so the coefficients are initially reported on the log scale.

```
# Model coefficients:
coef(mod_nb) # in log-scale
```

```
## (Intercept)   age_years
##  3.64420950  0.01042652
```

```
# Exponentiate the coefficients to interpret them on the original count scale:
exp(coef(mod_nb))
```

```
## (Intercept)   age_years
##   38.252522    1.010481
```

Exponentiated results: - `Intercept: 38.252522` - `age_years: 1.010481`

> Every additional year multiplies the expected parasite load by 1.01.

That's approximately a **1% increase** in expected parasite load **per year**.

Interpretation guide:

- exponentiated coefficient greater than 1: expected count increases
- exponentiated coefficient less than 1: expected count decreases
- exponentiated coefficient equal to 1: no change

```
age_percent_change <- (exp(coef(mod_nb)["age_years"]) - 1)*100
```

4. Predicted Values from the Negative Binomial Model

Create a sequence covering the observed age range.

```r
new_age <- tibble(
  age_years = seq(
    from = min(
      data$age_years,
      na.rm = TRUE
    ),
    to = max(
      data$age_years,
      na.rm = TRUE
    ),
    length.out = 100
  )
)
```

Use the fitted model to predict parasite load at each age.

```r
new_age$predicted_parasite_load <- predict(
  mod_nb,
  newdata = new_age,
  type = "response"
)

head(new_age)
```

```
## # A tibble: 6 x 2
##   age_years predicted_parasite_load
##       <dbl>                   <dbl>
## 1      3.4                     39.6
## 2      3.75                    39.8
## 3      4.09                    39.9
## 4      4.44                    40.1
## 5      4.79                    40.2
## 6      5.13                    40.4
```

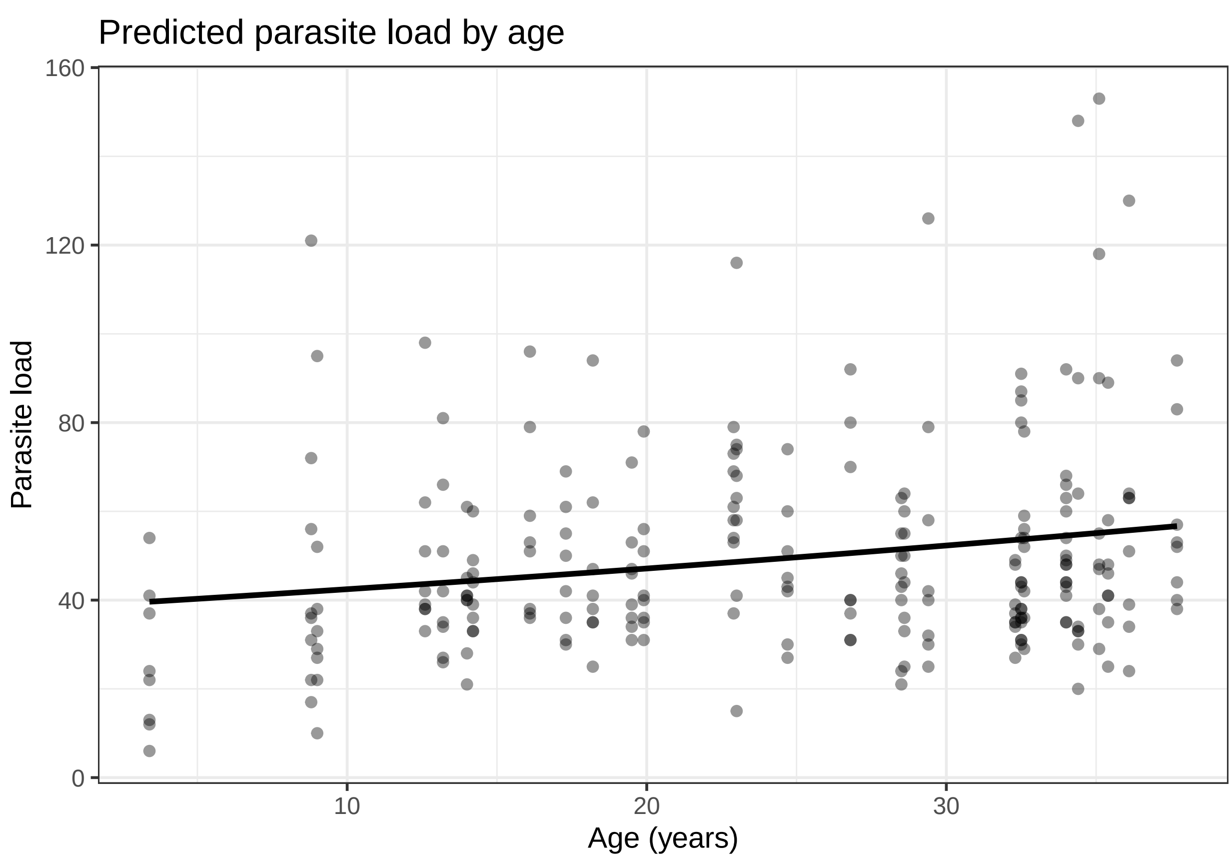
```r
tail(new_age)
```

```
## # A tibble: 6 x 2
##   age_years predicted_parasite_load
##       <dbl>                   <dbl>
## 1      36.0                    55.7
## 2      36.3                    55.9
## 3      36.7                    56.1
## 4      37.0                    56.3
## 5      37.4                    56.5
## 6      37.7                    56.7
```

Plot the observed counts and model predictions.

```r
ggplot(
  data,
  aes(
```

```
    x = age_years,
    y = parasite_load
  )
) +
  geom_point(alpha = 0.4) +
  geom_line(
    data = new_age,
    aes(
      x = age_years,
      y = predicted_parasite_load
    ),
    linewidth = 1
  ) +
  labs(
    x = "Age (years)",
    y = "Parasite load",
    title = "Predicted parasite load by age"
  ) +
  theme_bw()
```



Part 2: Binary Outcomes

5. What Is a Binary Outcome?

A binary variable has only two possible values.

Examples:

- injured / not injured
- sick / healthy
- present / absent
- alive / dead
- success / failure

In this dataset:

```
data %>%
  count(
    injury_status,
    injury_label
  )
```

```
## # A tibble: 2 x 3
##   injury_status injury_label     n
##           <dbl> <chr>        <int>
## 1             0 Not injured    220
## 2             1 Injured         20
```

The coding is:

- `0` = event did not occur
- `1` = event occurred

For a variable coded 0 and 1, the mean is the proportion of observations coded 1.

```
mean(data$injury_status, na.rm = TRUE)
```

```
## [1] 0.08333333
```

6. Logistic Regression

When the **response variable is binary**, we use a **logistic regression**.

In R, we use:

```
glm(..., family = "binomial")
```

Biological question:

> **Does parasite load predict the probability of injury?**

Response variable:

- `injury_status`, binary

Predictor variable:

- `parasite_load`, count

```
mod_logistic <- glm(
  injury_status ~ parasite_load,
  data = data,
  family = "binomial"
)

summary(mod_logistic)
```

```
##
## Call:
## glm(formula = injury_status ~ parasite_load, family = "binomial",
##     data = data)
##
## Coefficients:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -12.91314    2.57327  -5.018 5.22e-07 ***
## parasite_load   0.14891    0.03157   4.717 2.40e-06 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 137.681  on 239  degrees of freedom
## Residual deviance:  43.074  on 238  degrees of freedom
## AIC: 47.074
##
## Number of Fisher Scoring iterations: 8
```

This is very similar to the negative binomial output. The biggest difference is *what* we're predicting. Instead of predicting a **count**, we're predicting the **probability** that an event occurs (injured = 1, or 100%).

> Our biological question was: Does parasite load predict whether a gorilla is injured?

Response variable: `injury_status` (0 = not injured, 1 = injured)
Predictor: `parasite_load`

```
              Estimate
(Intercept)   -12.913
parasite_load   0.149
```

Like negative binomial regression, logistic regression is a generalized linear model (GLM), so the coefficients are not on the original scale; they are **on the log-odds scale**.

- **Intercept:** -12.913. This is the predicted log-odds of injury when **parasite load equals zero.**
- **Parasite load:** 0.149. The positive coefficient tells us: as parasite load increases, the probability of injury also increases.
- **p-value « 0.05:** so, there is strong evidence that parasite load is associated with injury status.
- **Fisher Scoring Iterations:** The Number of Fisher Scoring iterations is not a statistical result—it's simply information about how R fit the model.

Logistic regression (and other GLMs) cannot calculate the best-fitting line in one step. Instead, R starts with an **initial guess**, checks how well it fits the data, **adjusts** the coefficients, and **repeats** this process until the model no longer improves. Each adjustment is called a Fisher Scoring iteration.

In our case, R updated the model 8 times before it found the best-fitting coefficients.

6.1 Interpret Logistic Regression Coefficients

Logistic regression coefficients are reported on the log-odds scale.

```r
coef(mod_logistic)
```

```
##   (Intercept) parasite_load
##   -12.9131420     0.1489079
```

Exponentiating the coefficients gives odds ratios.

```r
exp(coef(mod_logistic))
```

```
##   (Intercept) parasite_load
##  2.465436e-06  1.160566e+00
```

Interpretation guide:

- odds ratio greater than 1: higher predictor values are associated with higher odds of the event
- odds ratio less than 1: higher predictor values are associated with lower odds of the event
- odds ratio equal to 1: no change in odds
- **Intercept:** the odds of injury when parasite load = 0. ust like in linear regression, the intercept corresponds to the prediction when the predictor equals zero. In reality, you usually don't interpret the intercept unless a value of 0 is **biologically meaningful**. Here it simply tells us that a gorilla with zero parasites has extremely low odds of being injured.
- **Estimate:** Parasite load: `exp(0.14891) = 1.1606`. For every one-unit increase in parasite load, the odds of injury are multiplied by 1.16.

Since 1.16 is 16% larger than 1, we usually say: > Each additional parasite is associated with approximately a 16% increase in the odds of injury.

Note that this is an **odds ratio.** Whereas **probability** tells us how likely an event is to happen, and ranges from 0 to 1 (or 0–100%), an **odds ratio** tells us how much the odds change when the predictor increases by one unit.

Calculate the odds ratio for parasite load.

7. Predicted Probabilities

Predicted probabilities are often easier to understand biologically than odds.

Create a sequence covering the observed parasite-load range.

```
new_parasites <- tibble(
  parasite_load = seq(
    from = min(
      data$parasite_load,
      na.rm = TRUE
    ),
    to = max(
      data$parasite_load,
      na.rm = TRUE
    ),
    length.out = 100
  )
)
```

Predict the probability of injury.

```
new_parasites$predicted_injury_probability <- predict(
  mod_logistic,
  newdata = new_parasites,
  type = "response"
)

head(new_parasites)
```

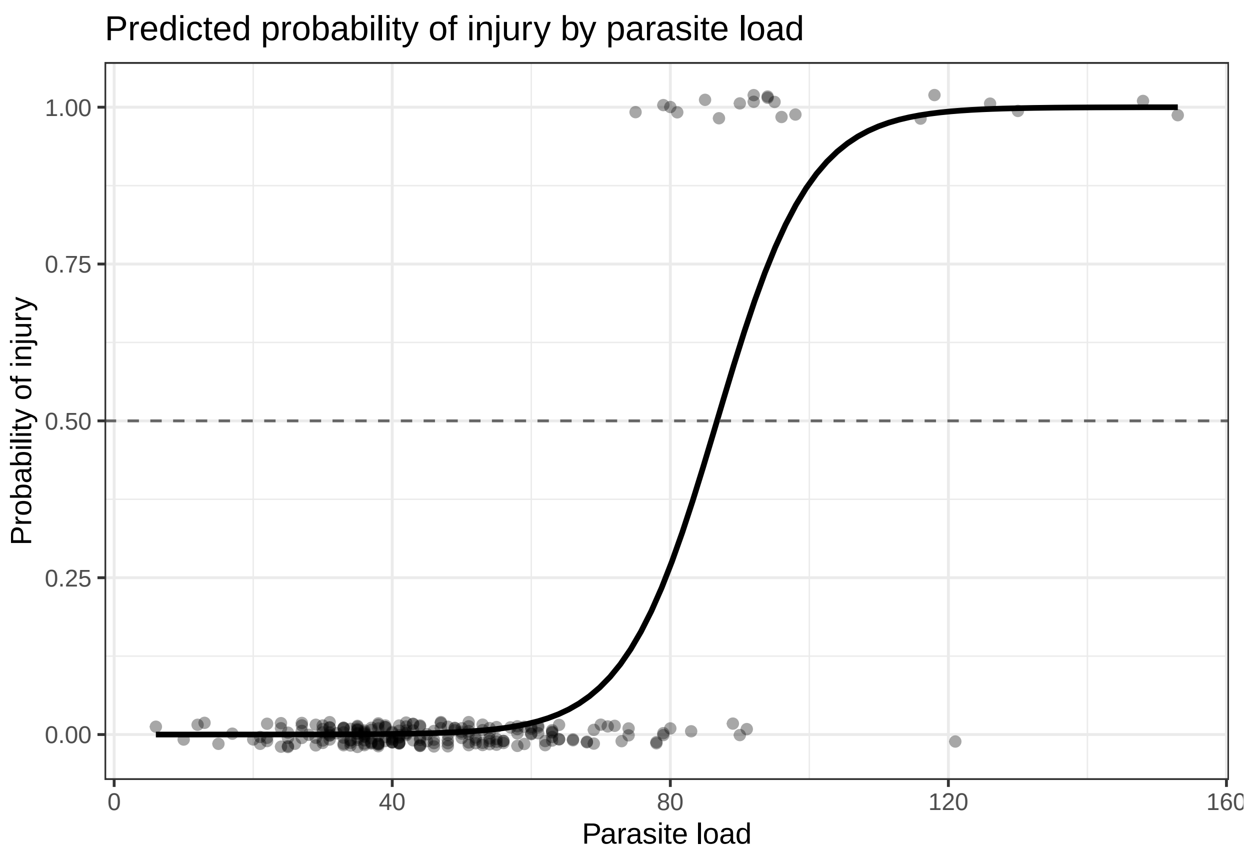
```
## # A tibble: 6 x 2
##   parasite_load predicted_injury_probability
##           <dbl>                        <dbl>
## 1          6                      0.00000602
## 2          7.48                   0.00000752
## 3          8.97                   0.00000937
## 4         10.5                    0.0000117
## 5         11.9                    0.0000146
## 6         13.4                    0.0000182
```

```
tail(new_parasites)
```

```
## # A tibble: 6 x 2
##   parasite_load predicted_injury_probability
##           <dbl>                        <dbl>
## 1          146.                         1.00
## 2          147.                         1.00
## 3          149.                         1.00
## 4          150.                         1.00
## 5          152.                         1.00
## 6          153                          1.00
```

Plot the observed binary outcomes and predicted probabilities.

```r
ggplot(data, aes(x = parasite_load, y = injury_status)) +
  geom_jitter(height = 0.02,
              width = 0,
              alpha = 0.35) +
  geom_line(data = new_parasites,
            aes(x = parasite_load,
                y = predicted_injury_probability), linewidth = 1) +
  geom_hline(yintercept = 0.5, linetype = "dashed", color = "grey40") +
  labs(
    x = "Parasite load",
    y = "Probability of injury",
    title = "Predicted probability of injury by parasite load"
  ) +
  theme_bw()
```



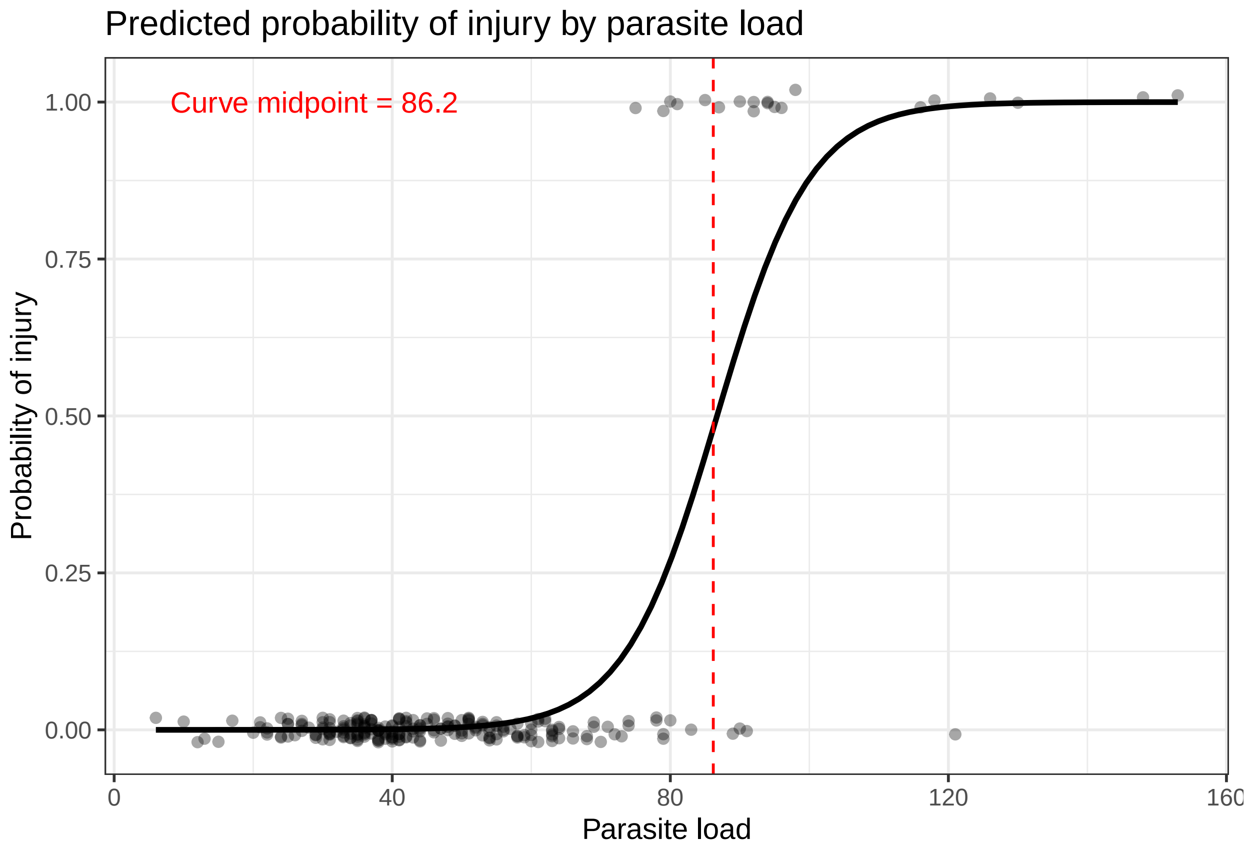
Now, with cutoff:

```r
cutoff <- new_parasites %>%
  slice_min(abs(predicted_injury_probability - 0.5))

# ^^^ this code finds the prediction where the model is closest to
# a 50% probability of injury. It does this by calculating how far
# each predicted probability is from 0.5 and then selecting the
# smallest difference.

# It uses the absolute distance from 0.5--
```

```r
# so it can select a value corresponding to a probability
# smaller or larger than 0.5.

cutoff
```

```
## # A tibble: 1 x 2
##   parasite_load predicted_injury_probability
##           <dbl>                        <dbl>
## 1          86.2                        0.480
```

```r
ggplot(data, aes(x = parasite_load, y = injury_status)) +
  geom_jitter(height = 0.02,
              width = 0,
              alpha = 0.35) +
  geom_line(data = new_parasites,
            aes(x = parasite_load,
                y = predicted_injury_probability), linewidth = 1) +
  geom_vline(xintercept = cutoff$parasite_load, linetype = "dashed", color = "red") +
  annotate("text", x = min(data$parasite_load), y = max(data$injury_status),
    label = paste0("Curve midpoint = ",
                   round(cutoff$parasite_load, 1)),
    hjust = -0.05,
    color = "red"
  ) +
  labs(
    x = "Parasite load",
    y = "Probability of injury",
    title = "Predicted probability of injury by parasite load"
  ) +
  theme_bw()
```



We asked the model, 'Which parasite load gives a probability closest to 50%?' The abs() function calculates how far every prediction is from 0.5, and slice_min() picks the one that's closest. That parasite load becomes our cutoff.

According to the logistic regression model, a gorilla with a parasite load of approximately 86 has about a 48% predicted probability of being injured. So you can say:

> When parasite load is (`cutoff`), the model predicts approximately a 50% probability of injury.

... or if the parasite load is, say, 96... then,
> *"based on its parasite load, this gorilla appears to be at relatively high risk"*.

That could happen for many reasons.

For example:

- Injured gorillas may become infected with more parasites.
- Gorillas with many parasites may be weaker and more likely to become injured.
- Poor health could increase both parasite load and injury risk.
- Another environmental factor could influence both.

As always, association is not causation. But, that information could help prioritize:

- veterinary monitoring,
- additional health examinations,
- or more frequent behavioral observations.

…and so on.

This 50% cutoff is what we call an **inflection point.** At this point:

- the curve changes from curving upward to curving downward,
- the predicted probability is 0.5,
- the slope of the curve is steepest, meaning **small changes in the predictor** produce the **largest changes in predicted probability**.

So mathematically, this inflection point is very meaningful.

> You can say: "According to our model, gorillas with parasite loads around 86 are at the transition between relatively low and relatively high predicted probabilities of injury."

Part 3: Relationships Between Categorical Variables

8. Contingency Tables

A contingency table displays the number of observations in each combination of two categorical variables.

Biological question:

> **Is injury status associated with sex?**

```r
injury_sex_table <- table(
  data$sex,
  data$injury_label
)

injury_sex_table
```

```
##
##     Injured Not injured
##   F       6         130
##   M      14          90
```

```r
# View proportions within each sex.
prop.table(
  injury_sex_table,
  margin = 1
)
```

```
##
##        Injured Not injured
##   F 0.04411765  0.95588235
##   M 0.13461538  0.86538462
```

9. Chi-Square Test of Independence

A chi-square test asks whether two categorical variables are associated. It first calculates **what the counts would look like if the variables were completely unrelated**, then those expected counts are **compared to the counts we actually observed**. If the observed counts are much farther from the expected counts than we would expect by random chance, the relationship is considered statistically significant.

The null hypothesis is:

> Injury status and sex are independent.

The alternative hypothesis is:

> Injury status and sex are associated.

```
# Add row and column totals.
addmargins(injury_sex_table)
```

```
##
##        Injured Not injured Sum
##   F          6         130 136
##   M         14          90 104
##   Sum       20         220 240
```

This is what is measured:

Sex	Injured	Not injured	Total
Female	6	130	**136**
Male	14	90	**104**
Total	**20**	**220**	**240**

We observed:

- 136 females
- 104 males
- 20 injured gorillas
- 220 uninjured gorillas

The chi-square test starts by assuming the null hypothesis: sex and injury are completely unrelated.

If that's true, then females and males should have the same injury rate: overall, 20 of 240 gorillas were injured.

So: `20 / 240 = 0.0833`, or `8.3%` of all gorillas were injured.

If sex *doesn't* matter, we'd **expect 8.3% of females** to be injured and **8.3% of males** to be injured.

```
chi_result <- chisq.test(
  injury_sex_table
)

chi_result
```

```
##
##  Pearson's Chi-squared test with Yates' continuity
##  correction
##
## data:  injury_sex_table
## X-squared = 5.1892, df = 1, p-value = 0.02273
```

Interpretation:

- a small p-value suggests an association
- a large p-value means we do not have strong evidence of an association

Biological interpretation:

> Injury status was associated with sex (chi-square test, p = 0.023).

10. Observed and Expected Counts

The chi-square test compares:

- **observed counts**: what we actually saw
- **expected counts**: what we would expect if the variables were unrelated

```
injury_sex_table
```

```
##
##     Injured Not injured
##   F       6         130
##   M      14          90
```

```
# Expected counts
chi_result$expected
```

```
##
##       Injured Not injured
##   F 11.333333   124.66667
##   M  8.666667    95.33333
```

Females: There are 136 females. If 8.3% should be injured: - 136 x 0.0833 = 11.33 injured females. - 136 - 11.33 = 124.67 not injured females.

Males: There are 104 females. If 8.3% should be injured: - 104 x 0.0833 = 8.67 injured males. - 104 - 8.67 = 95.33 not injured males.

Now, we compared:

	Observed	Expected
Female injured	6	11.3
Female not injured	130	124.7
Male injured	14	8.7

	Observed	Expected
Male not injured	90	95.3

You can immediately see the pattern:

- Fewer injured females than expected
- More injured males than expected

  The chi-square test then asks: are these differences larger than we'd expect from random chance?

If yes -> small p-value.
If no -> large p-value.

Large differences between observed and expected counts contribute more strongly to the chi-square statistic.

Conceptually, it's measuring: How different are the observed counts from the expected counts?

Small differences - small Xˆ2 - large p-value

Large differences - large Xˆ2 - small p-value

11. Fisher's Exact Test

The chi-square approximation may be unreliable when expected counts are very small.

Check the expected counts:

```
chi_result$expected
```

```
##
##       Injured Not injured
##   F 11.333333   124.66667
##   M  8.666667    95.33333
```

If one or more expected counts are below approximately 5, Fisher's exact test is often safer.

```
fisher.test(
  injury_sex_table
)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  injury_sex_table
## p-value = 0.01706
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.09040286 0.86418965
## sample estimates:
## odds ratio
##    0.29821
```

Fisher's exact test asks the same basic question:

> Are these two categorical variables associated?

It calculates an exact p-value rather than relying on the chi-square approximation.

Part 4: Choosing the Analysis

12. Summary

Response variable or question	Possible analysis
Continuous response	Linear regression
Count response	Negative binomial regression
Binary response	Logistic regression
Association between two categorical variables	Chi-square test
Two categorical variables with small expected counts	Fisher's exact test

Before choosing an analysis, ask:

1. What is the biological question?
2. What is the response variable?
3. What type of variable is the response?
4. Are we modeling an outcome or testing an association?
5. Does the model match the structure of the response variable?
6. Are the observations independent?

Skills Application – Lab

13. Lab Challenge

Work in your ongoing file:

`Lastname_Firstname_Data.Rmd`

Task List

1. Load your dataset and continue working in your script.
2. Identify one count, binary, or categorical variable.
3. Write a biological question using that variable.
4. Choose an appropriate analysis:
    - negative binomial regression
    - logistic regression
    - chi-square test
    - Fisher's exact test
5. Run the analysis.
6. Interpret the result biologically in 2-3 sentences.

7. Create predicted values or a plot only if it helps explain the result.

   Goal: Match the analysis to the response variable and complete the path from research question to biological insight.